

电子科技大学

University of Electronic Science and Technology of China

CPU Design Lab Report

Course Name：Application and Design of Digital Logic

Instructor:　　YanBo

Student Name：ChenYiChi

Student No.： [redacted]

Date Performed：　23, 04, 20224

格拉斯哥学院

Glasgow College，UESTC

1. Lab Description

- **Laboratory location：**

  Room 239 Scientific Research Building , Qingshuihe Campus

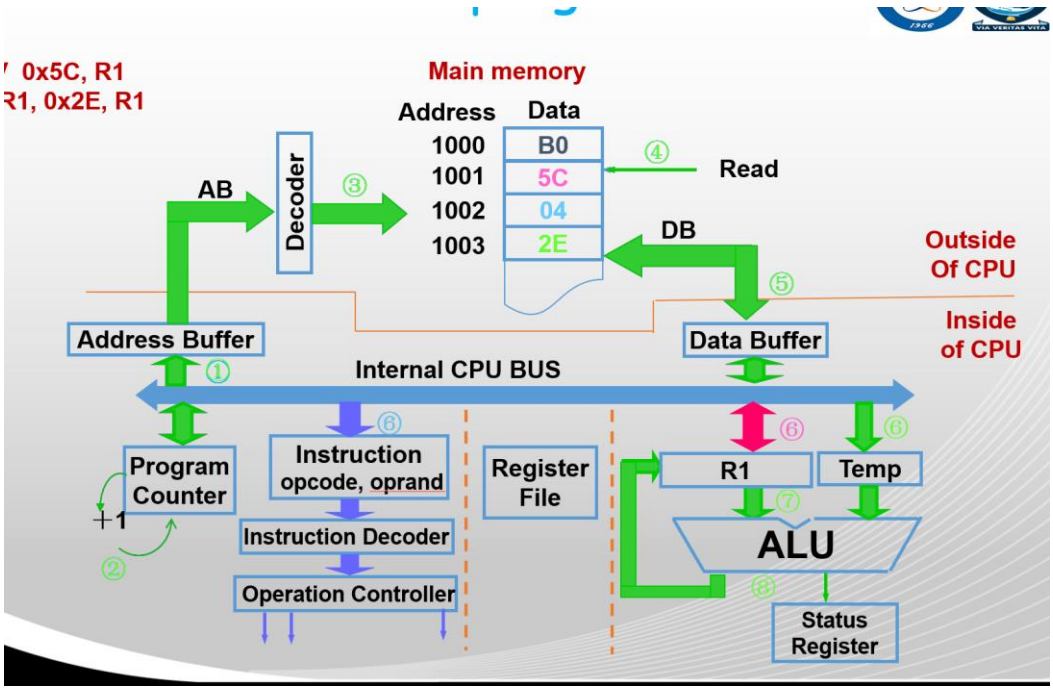
- **Lab hours:** 4 Classes (3 Hours) * 5

2. Specification

2.1 Instruction Set Design

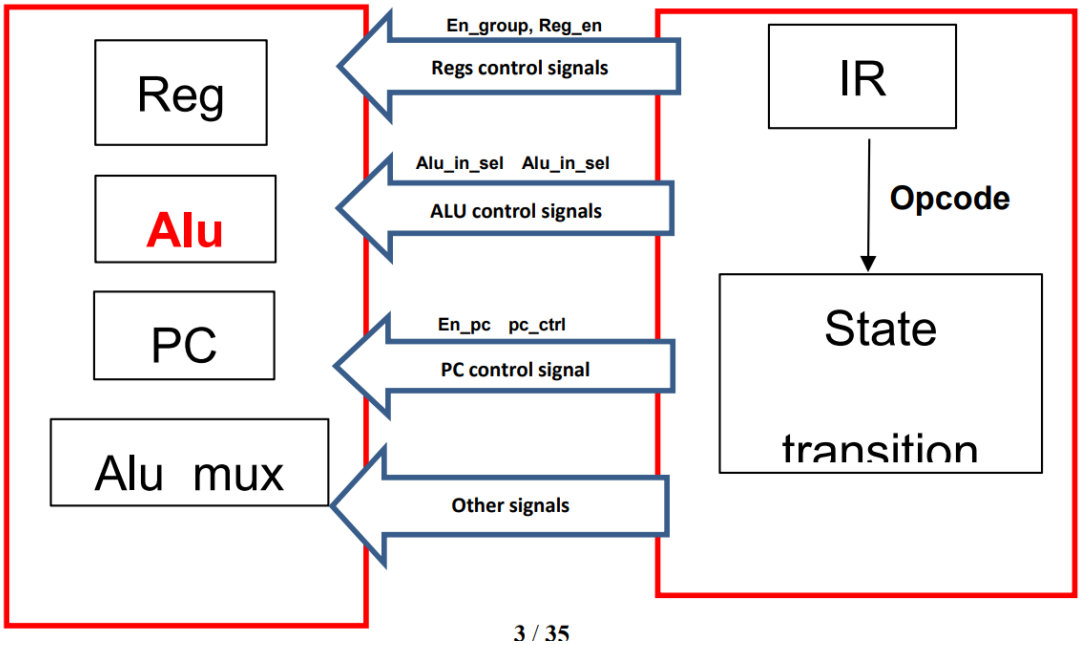
Opcode	Rd	Rs	Imme	assembly	function
0000	00	00	00000000	Mov R0#00	R0<= 0
0000	01	00	00001010	Mov R1#10	R1<=10
0000	00	00	00001000	Mov R0#08	R0<= 8
0010	01	00	00000001	Add R1#01	R1<=(R1)+1
0101	00	01	00000000	Sub R0 R1	R0<=(R0)-(R1)
1001	00	01	00000000	Or R0 R1	R0<=(R0) bit_or (R1)
0010	00	00	00000001	Add R0 #1	R0<=(R0)+1
0001	00	01	00000000	Move R0R1	R0<= R1
0011	00	01	00000000	Add R0 R1	R0<=(R0)+(R1)
0110	00	00	00000011	And R0 #3	R0<= (R0) bit_and (3)
1111	00	01	00000000	Xor R0 R1	R0<= (R0) Xor (R1)
1010	00	00	0111	Jump addr = 8'd7	PC<= 7

2.2 Working flow of the CPU design

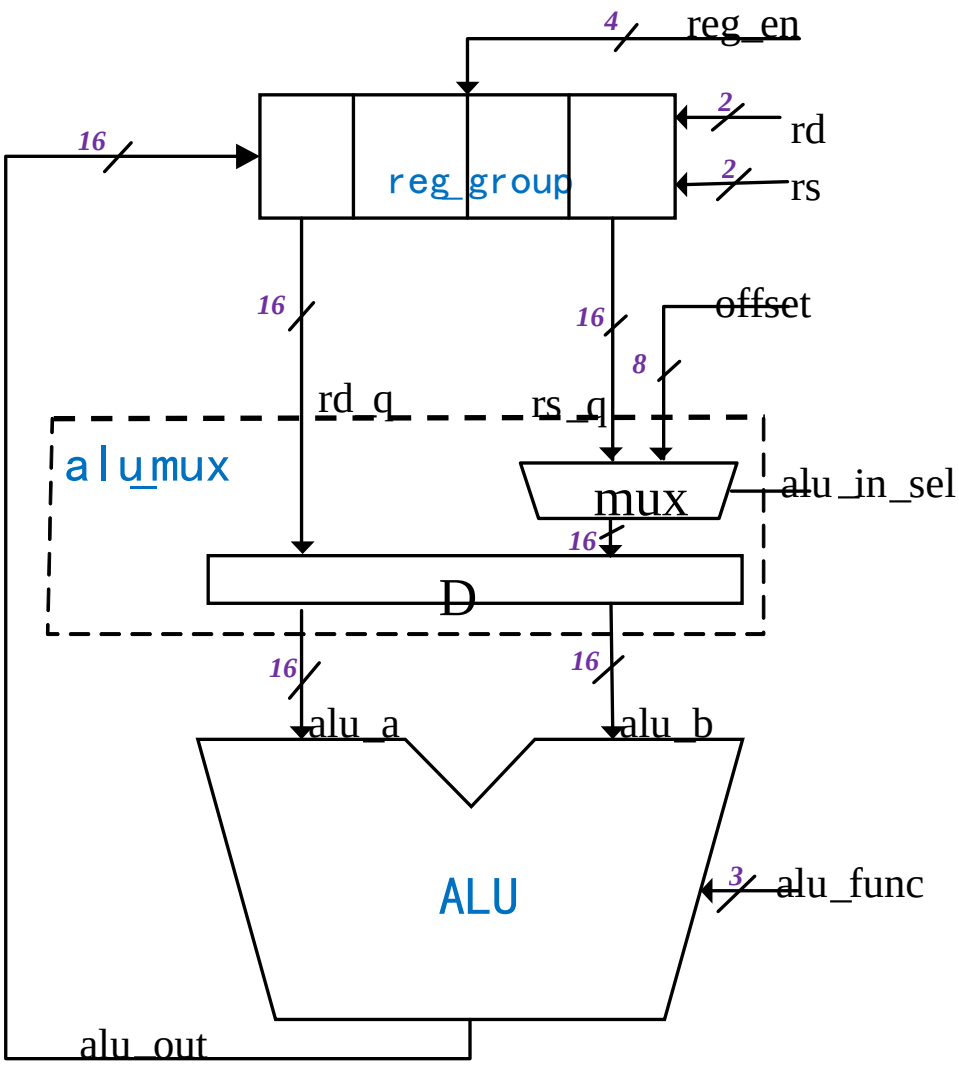


3. Architecture Design

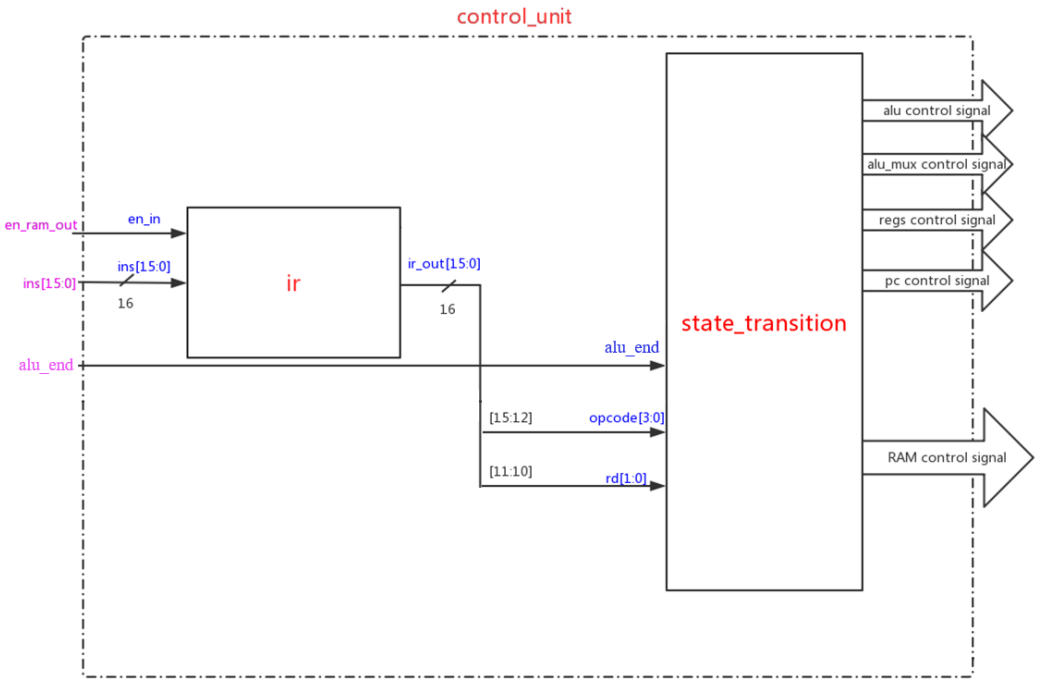
3.1Top level architecture



3.2 Datapath unit



3.3 Control unit



4. Implement

4.1 Completion procedures code

1) Datapath unit

```
module data_path (
        clk,rst,en_pc,pc_ctrl,offset,en_in,reg_en,alu_in_sel,alu_func,en_out,pc_out,rd,rs
);

input clk,rst,en_pc,en_in,alu_in_sel;
input [7:0] offset;
input [1:0] pc_ctrl,rd,rs;
input [3:0]    reg_en;
input [2:0] alu_func;
output    en_out;
output [15:0] pc_out;
```

```
wire en_out_group ,en_out_alu_mux;
wire [15:0] rd_q, rs_q ,alu_a ,alu_b ,alu_out;

pc pc1(
        .clk(clk),
        .rst(rst),
        .en_in(en_pc),
        .pc_ctrl(pc_ctrl),
        .offset_addr(offset),
        .pc_out(pc_out)
    );

reg_group reg_group1(
                .clk(clk),
                .rst(rst),
                .en_in(en_in),
                .reg_en(reg_en),
                .d_in(alu_out),
                .rd(rd),
                .rs(rs),
                .rd_q(rd_q),
                .rs_q(rs_q),
                .en_out(en_out_group)
            );

alu_mux alu_mux1(
                    .clk(clk),
                    .rst(rst),
                    .en_in(en_out_group),
                    .rd_q(rd_q),
                    .rs_q(rs_q),
                    .offset(offset),
                    .alu_in_sel(alu_in_sel),
                    .alu_a(alu_a),
                    .alu_b(alu_b),
                    .en_out(en_out_alu_mux)
                );

alu alu1 (
                    .clk(clk),
                    .rst(rst),
                    .en_in(en_out_alu_mux),
                    .alu_a(alu_a),
                    .alu_b(alu_b),
```

```
                .alu_func(alu_func),
                .en_out(en_out),
                .alu_out(alu_out )
            );

    endmodule
```

2) Control unit

```
module control_unit (

clk,rst,alu_end,ins,en_ram_in,en_ram_out,en_group,en_pc,reg_en,alu_in_sel,alu_func,pc_ctrl,ir_o
ut
);
input clk,rst,alu_end,en_ram_out;
input [15:0] ins;
output en_ram_in,en_group,en_pc,alu_in_sel;
output [3:0]    reg_en;
output [2:0]    alu_func;
output [1:0]    pc_ctrl;
output [15:0]     ir_out ;
ir ir1 (
            .clk(clk)  ,
            .rst(rst)     ,
            .ins(ins)     ,
            .en_in(en_ram_out),
            .ir_out(ir_out)
            );


state_transition state_transition1(
                .clk(clk) ,
                .rst(rst) ,
                .alu_end(alu_end) ,
                .rd(ir_out[11:10]) ,
                .opcode(ir_out[15:12])    ,
                .en_fetch(en_ram_in),
                .en_group_pulse(en_group),
                .en_pc(en_pc)    ,
                .pc_ctrl(pc_ctrl) ,
                .reg_en(reg_en) ,
                .alu_in_sel(alu_in_sel)    ,
                .alu_func(alu_func)
            );
endmodule
```

3) CPU

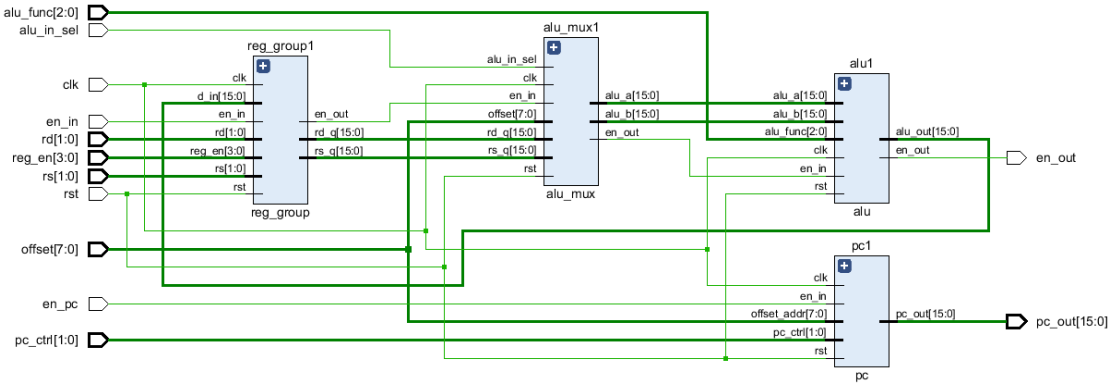
```
module control_unit (

clk,rst,alu_end,ins,en_ram_in,en_ram_out,en_group,en_pc,reg_en,alu_in_sel,alu_func,pc_ctr
l,ir_out
);
input clk,rst,alu_end,en_ram_out;
input [15:0] ins;
output en_ram_in,en_group,en_pc,alu_in_sel;
output [3:0]    reg_en;
output [2:0]    alu_func;
output [1:0]    pc_ctrl;
output [15:0]     ir_out ;
ir ir1 (
                        .clk(clk)  ,
                        .rst(rst)     ,
                        .ins(ins)     ,
                        .en_in(en_ram_out),
                        .ir_out(ir_out)
                        );

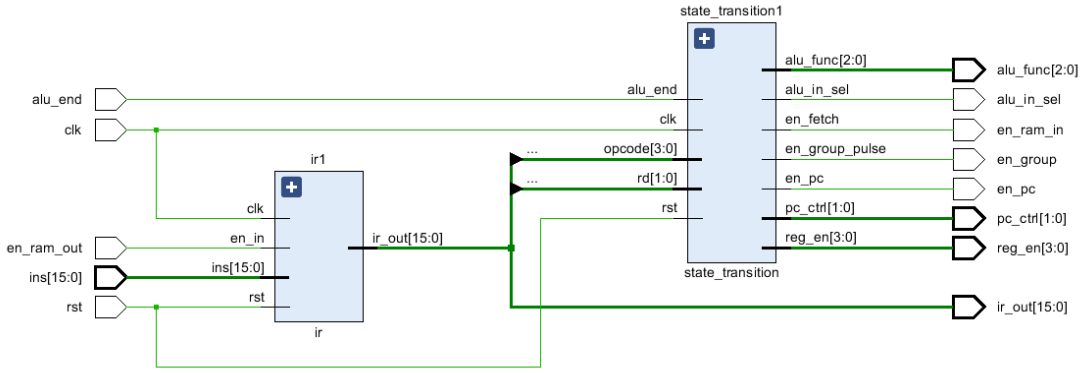

state_transition state_transition1(
                                .clk(clk) ,
                                .rst(rst) ,
                                .alu_end(alu_end) ,
                                .rd(ir_out[11:10]) ,
                                .opcode(ir_out[15:12])    ,
                                .en_fetch(en_ram_in),
                                .en_group_pulse(en_group),
                                .en_pc(en_pc)    ,
                                .pc_ctrl(pc_ctrl) ,
                                .reg_en(reg_en) ,
                                .alu_in_sel(alu_in_sel)    ,
                                .alu_func(alu_func)
                        );
endmodule
```

4.2 RTL Screenshot

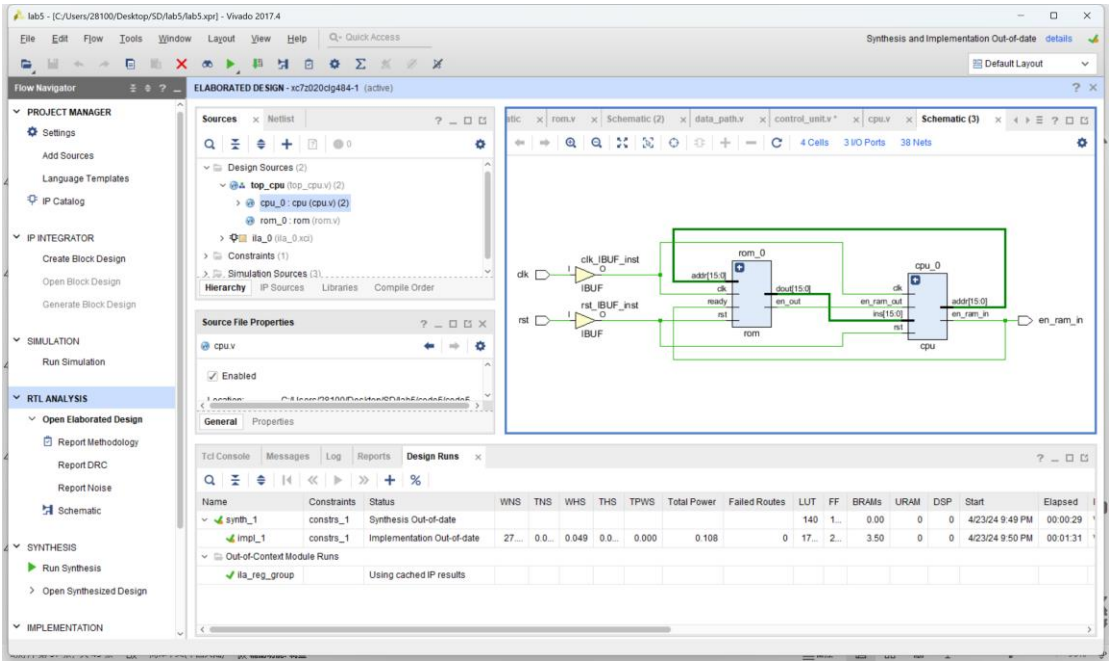
1) Datapath unit



2) Control unit



3) CPU



5. Testbench & Simulation& Analysis

5.1 Datapath unit

1) The code of testbench

```
`timescale 1ns / 1ps

module tb_alu();

wire en_out;
wire [15:0] alu_out;
reg clk,rst,en_in;
reg [15:0] alu_a;
reg [15:0] alu_b;
reg [2:0]  alu_func;

alu test_alu(
                        .clk        (clk),
                        .rst        (rst),
    .en_in      (en_in),
    .alu_a      (alu_a),
    .alu_b      (alu_b),
    .alu_func (alu_func),
    .en_out     (en_out),
    .alu_out    (alu_out)
                      );
parameter Tclk = 10;
initial begin
              clk=0;
              forever #(Tclk/2) clk=~clk;
end

initial begin
                                  rst=0;
   #(Tclk*4)            rst=1;
end

initial begin
                     alu_func = 3'b000   ;
    #(Tclk*8)     alu_func = 3'b001   ;
      //please add rest code here
    #(Tclk*4)     alu_func = 3'b010   ;
    #(Tclk*4)     alu_func = 3'b011   ;
    #(Tclk*4)     alu_func = 3'b100   ;
```

```
        #(Tclk*4)      alu_func = 3'b101    ;
        #(Tclk*4)      alu_func = 3'b110    ;
end

initial begin
                alu_a = 16'b0000000100000001;
                alu_b = 16'b0000000000010001;
end

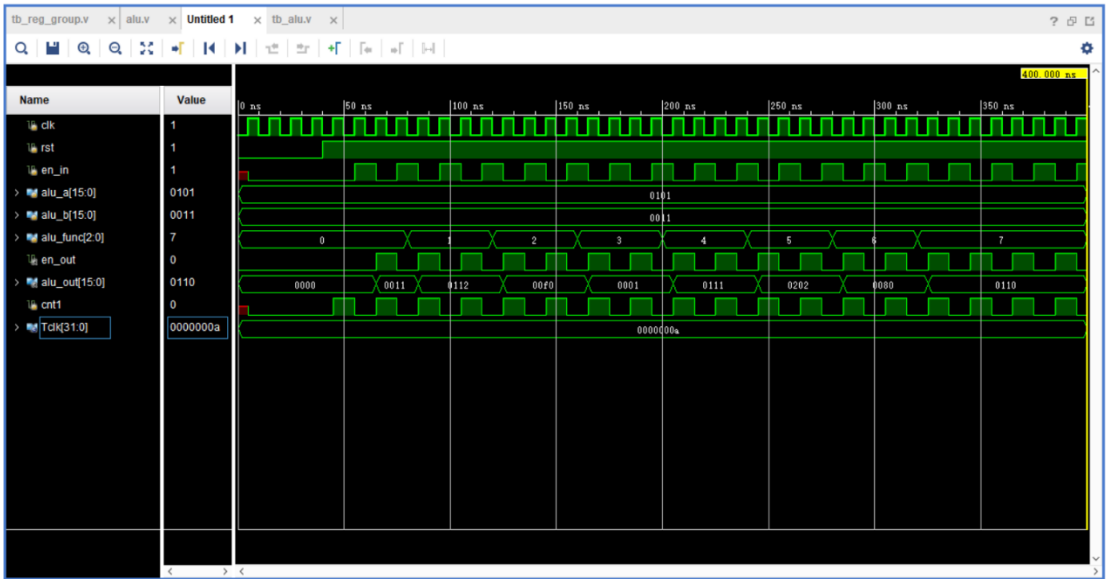
reg cnt1;
always @ (posedge clk) begin
        if(!rst)
                cnt1 <= 1'd0;
        else
                cnt1 <= cnt1+1;
end

always @ (posedge clk) begin
        if(!rst)
                en_in <= 1'b0;
        else if (cnt1==1'd1)
                en_in <= 1'b1;
        else
                en_in <= 1'b0;
end

initial begin
                #(Tclk*40)    $stop;
end

endmodule
```

2)simulation timing diagram



2) analysis

This test bench is used to verify the function of the alu part in datapath.

Initially, set the values of alu_a 16'b0000000100000001 and alu_b 16'b0000000000010001. After a short period, the reset signal was raised. Then I tested all the situations by altering the value of alu_func.

When alu_func= 001 execute ADD alu_out=alu_a+alu_b=0101+0011=0112

When alu_func= 010 execute SUB alu_out=alu_a--alu_b=0101-0011=00f0

When alu_func= 011 execute AND alu_out=alu_a&alu_b=0101&0011=0001

When alu_func= 100 execute SUB alu_out=alu_a|alu_b=0101|0011=0111

When alu_func= 101 execute LEFTSHIFT alu_out=0202

When alu_func= 1110 execute RightSHIFT alu_out=0080

Besides, the cnt accumulates if the value of clock is high.

5.2 Control unit

1) The code of testbench

```
`timescale 1ns / 1ps
module tb_state_transition();

reg clk,rst;
reg alu_end;
reg [1:0] rd;
reg [3:0] opcode;
wire en_fetch;
wire en_group;
wire en_pc;
wire [1:0] pc_ctrl;
wire [3:0] reg_en;
wire alu_in_sel;
wire [2:0] alu_func;
```

```
state_transition state_transition1(
                                .clk(clk) ,
                                .rst(rst) ,
                                .alu_end(alu_end) ,
                                .rd(rd) ,
                                .opcode(opcode)    ,
                                .en_fetch(en_fetch),
                                .en_group_pulse(en_group),
                                .en_pc(en_pc)    ,
                                .pc_ctrl(pc_ctrl) ,
                                .reg_en(reg_en) ,
                                .alu_in_sel(alu_in_sel)    ,
                                .alu_func(alu_func)
                        );


parameter Tclk = 10;

initial begin
                clk=1;
                forever #(Tclk/2) clk=~clk;
        end

initial begin
                                rst=0;
            #(Tclk*3)    rst=1;
        end

reg [2:0]cnt1;
always @ (posedge clk) begin
    if(!rst)
        cnt1 <= 3'd1;                  // When being reset, the cnt1 is initialized as 1 to align alu_end
to posedge of first execution state
    else
        cnt1 <= cnt1+1;                  // cnt1 means that alu_end should be pulled up periodically
end

always @ (posedge clk) begin
    if(!rst)                            // Pull up alu_end every 5 clocks, this block should involve
cnt1
        alu_end <= 1'b0;
    if(cnt1==3'd4)
        begin
```

```
            alu_end <= 1'b1;
            cnt1 <= 3'd0;
        end
    else
        alu_end <= 1'b0;
    // Please add your code here
end

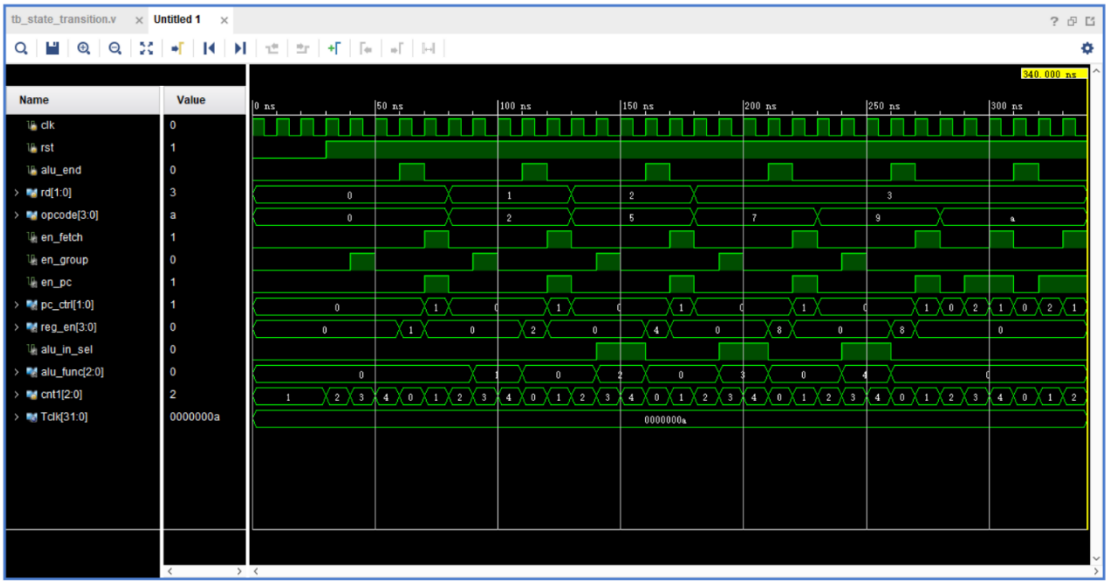
initial begin
                            rd = 2'b00;
        #(Tclk*3)       rd = 2'b00;     // 3 clocks for aligning rd at Fetch state
        #(Tclk*5)       rd = 2'b01;
        #(Tclk*5)       rd = 2'b10;
        #(Tclk*5)       rd = 2'b11;
      end

initial begin
                            opcode=4'b0000;
        #(Tclk*3)       opcode=4'b0000;
        #(Tclk*5)       opcode=4'b0010;
        #(Tclk*5)       opcode=4'b0101;
        #(Tclk*5)       opcode=4'b0111;
        #(Tclk*5)       opcode=4'b1001;
        #(Tclk*5)       opcode=4'b1010;
        // Please add your code here
      end

initial begin
        #(Tclk*34)  $stop;
end

endmodule
```

2) Simulation timing diagram



3) Analysis

In the simulation, I mainly focused on verifying the function of the control unit by changing the opcode which can then change the state of state transition.

When en_group=1and op_code=4'b0000=0 alu_func=0:

When en_group=1and op_code=4'b0010=2 alu_func=1:

When en_group=1and op_code=4'b0101=5 alu_func=2:

When en_group=1and op_code=4'b0111=7 alu_func=3:

When en_group=1and op_code=4'b1001=9 alu_func=4:

As a result, the functionality of the control unit has been successfully verified

5.3 CPU

1) The code of testbench

```
`timescale 1ns / 1ps


module tb_cpu();
wire en_ram_in;
wire [15:0] addr;
reg     clk,rst,en_ram_out;
reg   [15:0] ins;

cpu test_cpu(
      .clk (clk),
      .rst (rst),
      .addr (addr),
      .ins (ins),
      .en_ram_in (en_ram_in),
      .en_ram_out (en_ram_out)
```

```
);
parameter Tclk = 10;
reg [15:0] instruction_memory[0:255];

initial begin
    // Initialize instruction memory
    instruction_memory[0] = 16'b0000_0000_0000_0000;
    instruction_memory[1] = 16'b0000_0000_0000_1000; // MOV R0 #08
    instruction_memory[2] = 16'b0000_0100_0000_0010; // MOV R1 #02
    instruction_memory[3] = 16'b0010_0100_0000_0001; // ADD R1 #01
    instruction_memory[4] = 16'b0101_0001_0000_0000; // SUB R0 R1
    instruction_memory[5] = 16'b1001_0001_0000_0000; // OR    R0 R1
    instruction_memory[6] = 16'b0010_0000_0000_0001; // ADD R0 #01
    instruction_memory[7] = 16'b1011_0001_0000_0000;
    instruction_memory[8] = 16'b1010_0000_0000_0111; // Jump to addr 7
end

initial begin
        // Define clk、
        clk=1;
        forever #(Tclk/2) clk=~clk;
end
initial begin
        // Define rst
        rst=0;
        #(Tclk*5)       rst=1;
end
initial begin
        // Define en_ram_out
        en_ram_out = 1;

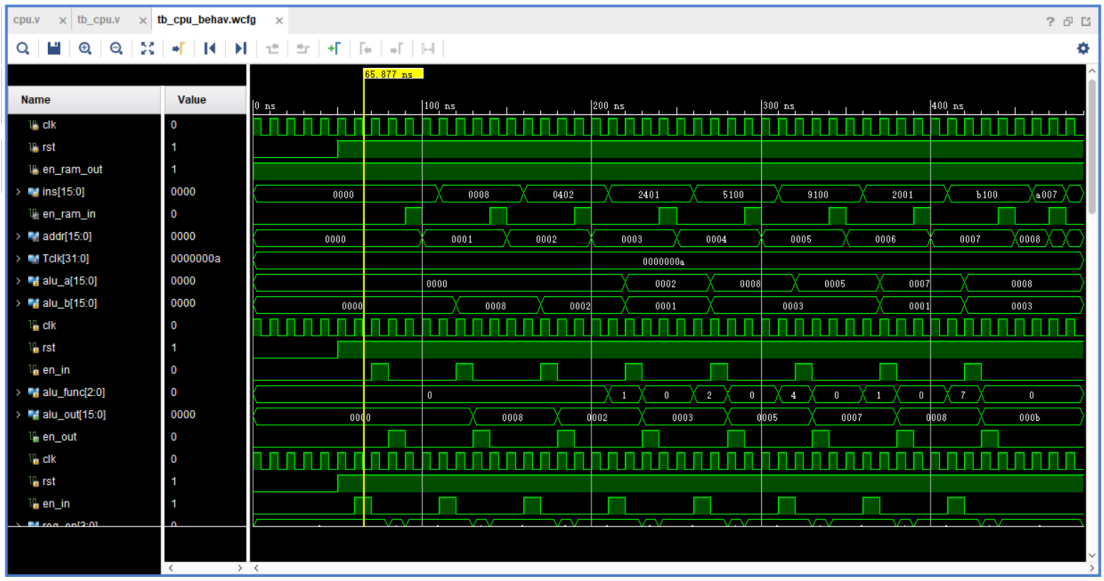
end

always @(posedge clk) begin
    if (rst && en_ram_out) begin
        // Fetch instruction from memory indexed by 'addr'
        ins = instruction_memory[addr];    // addr needs to be adjusted if it exceeds 16 bits
    end
end
initial begin
        // Define en_ram_out
        ins = 16'b0000_0000_0000_0000;
end
initial begin
```

```
    #(Tclk*49) $stop; // Stop the simulation after some cycles
end

endmodule
```

2) Simulation timing diagram



3) Analysis

This simulation is to verify the function of cpu.

When instruction_memory[1] = 16'b0000_0000_0000_1000 //MOV R0 #08 alu_out=0008;

When instruction_memory[2] = 16'b0000_0100_0000_0010; // MOV R1 #02 alu_out=0002;

When instruction_memory[3] = 16'b0010_0100_0000_0001; // ADD R1 #01 alu_out=0003;

When instruction_memory[4]=16'b0101_0001_0000_0000;//SUBR0R1 alu_out=0008-0003=0005

When instruction_memory[5] = 16'b1001_0001_0000_0000; // OR R0 R1 alu_out=0005|0003=0007

When instruction_memory[6] = 16'b0010_0000_0000_0001; // ADD R0 #01 alu_out=0008

When instruction_memory[7] = 16'b1011_0001_0000_0000;// XOR R0 R1 alu_out=0008 xor 0003=000b

When instruction_memory[8] = 16'b1010_0000_0000_0111; // Jump to addr 7 adddr=7

The code is verified.

6. Your Suggestion for This Lab (Course)

No suggestion.

The end